



MK : Proyek 1

PENGUJIAN PERANGKAT LUNAK

Inggrid Y. Risca P., S.S.T., M.Tr.T.

Tujuan

- Memahami teknik pengujian komponen dan pengujian sistem yang digunakan untuk menemukan kesalahan program
- Memahami penggunaan teknik pengujian black box dan pengujian struktural
- Mengetahui cara pembuatan test case dan pemilihan data uji

Pengujian

- Pengujian → proses eksekusi aplikasi, dengan maksud menemukan kesalahan
- Test case yang baik → test case yang memiliki probabilitas tinggi untuk menemukan kesalahan yang belum pernah ditemukan sebelumnya
- Pengujian yang sukses → pengujian yang mengungkap semua kesalahan yang belum pernah ditemukan sebelumnya

Komponen dan Sistem Tes

- **Komponen Test :**

- Test terhadap komponen **perangkat lunak** secara terpisah yang merupakan tanggung jawab dari developer

- **Sistem Test :**

- Test terhadap **keseluruhan sistem & subsistem** merupakan tanggung jawab dari tim tester yang independent → QA(Quality Assurance)

Pengujian Integrasi (Integration Test)

- Ketika komponen secara individu telah teruji, mereka harus diintegrasikan untuk membentuk sistem yang lengkap dan perlu diuji interaksinya
- Untuk melokalisir masalah, lebih baik integrasi dilakukan secara bertahap
- Pengujian Top-Down → menguji modul dari tingkat paling atas (level utama) ke modul yang lebih rendah secara bertahap.
- Pengujian Bottom Up → Dimulai dari komponen terkecil, modul sampai keseluruhan sistem

Contoh Pengujian Top-Down & Buttom Up

Pengujian Top-Down

- 1) Struktur modul Sistem Pemesanan Online → Modul Utama (Main), Modul Login, Modul Pilih Produk, Modul Pembayaran
- 2) Uji Modul Utama terlebih dahulu → Jalankan alur utama (misalnya: masuk → pilih produk → bayar)
- 3) Modul Login → hanya menampilkan “Login berhasil
- 4) Pilih Produk → hanya menampilkan “Produk dipilih”
- 5) Pembayaran → hanya menampilkan “Pembayaran sukses”

Pengujian Buttom-up

- 1) Uji Modul Login secara terpisah
- 2) Uji Modul Pilih Produk secara terpisah
- 3) Uji Modul Pembayaran secara terpisah
- 4) Uji Login + Pilih Produk
- 5) Uji Login + Pilih Produk + Pembayaran

Pengujian Cacat & Pengujian Validasi

- **Pengujian cacat (defect) :**

- Untuk menemukan cacat sistem, dimana sistem berjalan tidak semestinya
- Dianggap berhasil jika menemukan sistem berlaku tidak benar

- **Pengujian Validasi :**

- Membuktikan bahwa sistem memenuhi spesifikasinya
- Test yang sukses menunjukkan bahwa sistem berjalan sesuai dengan yang diinginkan
- Pengujian yang baik harus dapat ditelusuri ke requirement customer

Pengujian **Black Box** dan **White Box**

- **Pengujian Fungsional (black box) :**

- Sistem dianggap sebagai kotak hitam yang perilakunya **hanya dapat ditentukan dengan mempelajari input dan outputnya**
- Metode yang paling umum digunakan untuk release/acceptance testing karena **merepresentasikan perilaku user/customer**

- **Pengujian Struktural (white box)**

- Pengujian diturunkan dari pengetahuan mengenai kode dan implementasinya
- Test case disusun untuk memastikan bahwa sebanyak mungkin statemen pada program telah diuji

Black Box VS White Box

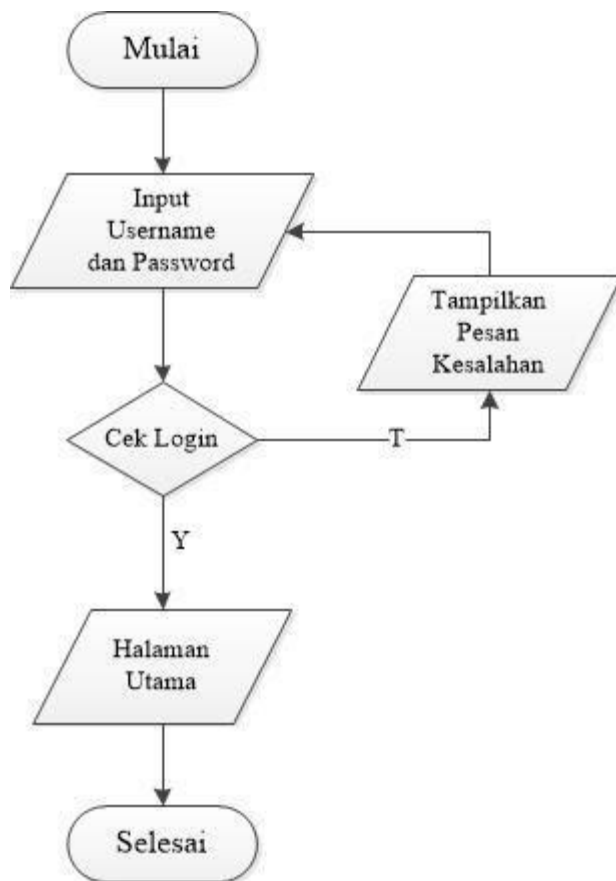
White Box

- Dilakukan oleh penguji yang mengetahui tentang QA.
- Pengujian aplikasi dalam hal keamanan dan kinerja program (termasuk pengujian kode, perencanaan implementasi, keamanan, aliran data, kegagalan perangkat lunak).
- Dilakukan seiring dengan tahapan pengembangan software/pada tahap testing.

Black Box

- Dilakukan oleh auditor independen
- Melakukan tes berdasarkan apa yang dilihat, hanya fokus pada fungsionalitas dan output.
- Pengujian cenderung berfokus pada standar desain perangkat lunak dan respons ketika ada celah/kerentanan dalam program aplikasi setelah pengujian kotak putih.
- Dilakukan setelah white box testing.

CONTOH PENGUJIAN WHITE BOX



- Menyocokkan alur sistem dengan flowchart fungsinya
- Dari hasil masing-masing penyocokan dapat dibuat tabel pengujiannya

1. Method uploadAssignment()

No	No. Jalur	Data Input	Expected Result	Result	Status
1.	1 – 2 – 3 – 4 – 5	File document berextensi pdf atau doc	File terupload dan menampilkan pesan berhasil upload	File Berhasil terupload dan menampilkan pemberitahuan bahwa upload berhasil	Valid
2.	1 – 2 – 5	Tidak ada file document yang dipilih tapi menekan tombol upload	Menampilkan pemberitahuan dan Kembali ke halaman upload	Menampilkan pemberitahuan tetapi dan kembali ke halaman upload	Valid
3	1 – 3 – 4 – 5	File yang di upload tidak berekstensi pdf atau doc	Menampilkan pemberitahuan dan Kembali ke halaman upload	Menampilkan pemberitahuan cek file kembali dan kembali kehalaman upload	Valid

Tabel 5. Tabel Pengujian Unit

CONTOH PENGUJIAN BLACK BOX

Tabel 1 Black Box Testing

N o	Skenario Pengujian	Test Case	Hasil yang di harapkan	Hasil Pengujian	Keterangan
1	ID dan Password tidak di isi, kemudian klik tombol Login	ID dan Password kosong	Sistem akan menolak akses login dan menampilkan pesan "ID atau Password salah"	Sesuai harapan	Valid
2	ID di isi dan Password tidak di isi, kemudian klik tombol Login	ID: numuthia dan Password kosong	Sistem akan menolak akses login dan menampilkan pesan "ID atau Password salah"	Sesuai harapan	Valid
3	ID tidak di isi dan Password di isi, kemudian klik tombol Login	ID kosong dan Password: 140918	Sistem akan menolak akses login dan menampilkan pesan "ID atau Password salah"	Sesuai harapan	Valid

4	Mengetikkan kondisi salah pada ID atau Password, kemudian klik tombol Login	ID: numuth dan Password: 140920	Sistem akan menolak akses login dan menampilkan pesan "ID atau Password salah"	Sesuai harapan	Valid
5	Mengetikkan ID dan Password dengan data yang benar, kemudian klik tombol Login	ID: numuthia Password: 140918	Sistem akan menerima akses login dan menampilkan Menu Utama.	Sesuai harapan	Valid

Pengujian Performa



- dirancang untuk menjamin bahwa sistem dapat memproses beban/task yang diberikan sesuai dengan spesifikasinya
- menjamin bahwa ketika sistem gagal karena beban yang berat (stress), perilakunya tidak menyebabkan kegagalan/kerugian kepada user atau sistem selanjutnya
- Pengujian performa harus dilakukan sampai ambang batas
→ Ambang batas (batas input yang mungkin terjadi / kegagalan sistem)
- Contoh : 200 transaksi per detik, 100 terminal secara bersamaan
- Pengukuran pengujian performa bisa menggunakan **latency**

$$\text{Latency} = \text{Waktu Diterima} - \text{Waktu Dikirim}$$

UAT (*User Acceptance Test*)

- untuk pastikan produk yang dikembangkan sesuai dengan kebutuhan dan harapan *end user*.
- Pada tahapan ini, *user* diberi kesempatan untuk berinteraksi langsung dengan perangkat lunak sebelum peluncuran resmi, yang memungkinkan mereka mengidentifikasi aplikasi jika ada fitur yang kurang atau adanya *bug*.
- Tujuan utama UAT → untuk memastikan bahwa sistem (yang telah melewati berbagai tahap pengujian) dapat beroperasi dengan efektif di dunia nyata dan sesuai dengan kriteria bisnis.

UAT (User Acceptance Test)

- UAT juga berguna dalam mengidentifikasi potensi masalah yang belum terungkap pada tahapan pengujian sebelumnya.
- Tanpa adanya UAT, risiko merilis perangkat lunak yang masih mengandung *bug* atau tidak sesuai dengan tujuan *end user* lebih besar, hingga akhirnya harus menambah biaya maintenance software.
- UAT berperan : validasi kebutuhan user, identifikasi bug, kurangi resiko & biaya, tingkatkan kepuasan user
- Contoh: <https://ejournal.uin-malang.ac.id/index.php/saintek/article/view/36225/13636>

UAT (User Acceptance Test)



- Pelaksanaan UAT melibatkan beberapa pihak yang memiliki kepentingan terhadap perangkat lunak yang dikembangkan, termasuk *end user*, *business stakeholder*, tim QA, dan *developer*.
- Namun, tanggung jawab utama pelaksanaan UAT ada di *end user*.
- Selain *end user*, ahli fungsional internal (seperti QA dan *developer*) juga memainkan peran penting dalam jalannya UAT.
- Mereka membantu membentuk siklus UAT dan manajemen pengujian, serta menginterpretasikan hasil dari pengujian tersebut.

Jenis UAT

1. **Alpha testing** → pengujian akhir sebelum perangkat lunak diluncurkan untuk pengguna secara umum.
2. **Beta testing** → pengujian pengguna berlangsung di lokasi end user
3. Link Contoh UAT : <https://ejournal.uin-suka.ac.id/saintek/ijid/article/view/4297/2737>

Table 3. Alpha Testing

Alpha Testing Result			
<i>Component</i>	<i>Scenario</i>	<i>Expected Output</i>	<i>Result</i>
Home Page	Learning Menu	Go to the Learning page	Satisfied
	Play Menu	Go to play page	Satisfied

Alpha Testing

Ada 2 fase :

- 1) Perangkat lunak **diuji oleh developer di lingkungan internal developer.**
- 2) **Perangkat lunak ini diserahkan kepada staf QA (Quality assurance),** untuk pengujian tambahan dalam lingkungan yang mirip dengan penggunaan yang dimaksudkan. Hal ini untuk mensimulasikan suasana atau lingkungan pengujian yang sebenarnya sehingga ketika sistem tersebut dipasang, sudah tidak terjadi kegagalan maupun cacat sistem secara real

Beta Testing



- Pengujian beta → **pengujian pengguna berlangsung di lokasi end user** untuk memvalidasi kegunaan, fungsi, kompatibilitas, dan uji reliabilitas dari perangkat lunak yang dibuat = uji lapangan.

Table 4. Beta Testing

Questions	Very Agree	Agree	Disagree	Very Disagree
Edugame Arabic was designed with an interesting theme	42.9%	57.1%	0%	0%
Edugame Arabic has a good and satisfying visual design	33.3%	66.7%	0%	0%
Edugame Arabic can convey material well	42.9%	57.1%	0%	0%
Edugame Arabic could help in learning the Arabic language	52.4%	47.6%	0%	0%

Usability Testing

- Usability merupakan bagian dari UX.
- Usability digunakan sebagai evaluasi kemudahan penggunaan sistem
- Usability → persepsi end user tentang bagaimana seseorang dapat secara efektif, efisien dan puas dalam menyelesaikan tugas saat menggunakan aplikasi
- Usability Testing adalah metode untuk menguji:
 - Kemudahan penggunaan sistem
 - Efektivitas fitur
 - Kepuasan pengguna

Metode Usability Testing

- **Task-based testing** → user menjalankan tugas tertentu
- **Think aloud** → user menjelaskan saat menggunakan sistem
- **Kuesioner** → mengukur kepuasan
- **Observasi langsung**

Indikator Penilaian Usability Testing

- Waktu penyelesaian tugas
- Tingkat keberhasilan (%)
- Jumlah error
- Skor kepuasan (misal skala 1–5) → skala likret

Contoh Kuesioner Pengujian Usability Testing



- <https://repository.ub.ac.id/id/eprint/11820/6/LAMPIRAN.pdf>
- [https://repository.pradita.ac.id/id/eprint/188/1/Cicilia%20Hera%20Noverda%20\(SI\).pdf](https://repository.pradita.ac.id/id/eprint/188/1/Cicilia%20Hera%20Noverda%20(SI).pdf)
- <https://repository.ub.ac.id/id/eprint/186493/6/Rihadatul%20%E2%80%98Aisy.pdf>
- <https://media.neliti.com/media/publications/228045-pengujian-usability-website-menggunakan-8af9e315.pdf>
- <https://jurnal.kolibi.org/index.php/scientica/article/view/4653/4390>

Contoh Kuesioner

Pengujian **Usability Testing**

No	Pernyataan	Skor (1–5)
1	Saya ingin menggunakan sistem ini secara rutin	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
2	Sistem ini terasa rumit digunakan	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
3	Sistem mudah digunakan	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
4	Saya membutuhkan bantuan saat menggunakan sistem	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
5	Fitur dalam sistem terintegrasi dengan baik	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
6	Sistem ini membingungkan	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
7	Orang lain akan cepat memahami sistem ini	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
8	Sistem terasa tidak praktis digunakan	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
9	Saya percaya diri menggunakan sistem ini	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
10	Saya perlu belajar banyak sebelum menggunakan sistem	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

Contoh Kuesioner & Menghitung Skala Likret

Pengujian Usability Testing

Skala	Keterangan
1	Sangat Tidak Setuju
2	Tidak Setuju
3	Netral
4	Setuju
5	Sangat Setuju

Rumus

Skala Likret

Skor Tertinggi = Skor Tertinggi *Likert* x Total Responden

Skor Terendah = Skor Terendah *Llikert* x Total Responden

Rumus Index% = $Total\ skor / X * 100$

Perbedaan UAT & SIT

- **UAT** → jenis pengujian yang dilakukan untuk **memastikan software telah dibangun sesuai dengan persyaratan user dan bisnis**. Tahap ini menjadi akhir dari proses pengujian dan dilakukan sebelum *software* dirilis ke pasar atau diimplementasikan di lingkungan produksi.
- **SIT (*System Integration Testing*)** → jenis pengujian yang dilakukan untuk **memeriksa interaksi antara berbagai komponen sistem** dan memastikan semuanya bekerja dengan baik bersama-sama. **Tahap ini dilakukan setelah pengujian unit dan sebelum UAT.**

Perbedaan UAT VS SIT

Aspek	UAT (User Acceptance Testing)	SIT (System Integration Testing)
Tujuan	Memastikan sistem memenuhi kebutuhan pengguna akhir	Memastikan integrasi komponen sistem berjalan baik
Pelaku	Pengguna akhir atau pemangku kepentingan bisnis	Tim pengembang atau tim QA
Cakupan	Fungsionalitas keseluruhan sistem	Interaksi antar komponen dan modul sistem
Lingkungan	Mirip dengan lingkungan produksi	Lingkungan terkontrol khusus untuk pengujian
Kriteria Keberhasilan	Memenuhi kebutuhan dan ekspektasi pengguna	Tidak ada masalah integrasi antar komponen

Pengujian White Box VS Black Box VS Integration Test VS UAT



Aspek	White Box	Black Box	Integration Testing	UAT
Fokus	Logika internal / kode	Fungsi (input-output)	Hubungan antar modul	Kebutuhan pengguna
Tujuan	Validasi kebenaran kode	Validasi fitur sesuai spesifikasi	Memastikan modul terintegrasi dengan baik	Memastikan sistem siap digunakan
Akses Kode	Wajib	Tidak perlu	Tidak wajib	Tidak perlu
Pelaku	Developer	Tester / QA	Developer / Tester	User / Pengguna akhir
Level Pengujian	Unit	Sistem	Antar modul	Akhir (sebelum rilis)
Contoh	Uji <code>if-else</code> , loop	Uji form login	Login terhubung ke pembayaran	User mencoba sistem sesuai skenario nyata

Ilustrasi **UAT**:

Case Study : **APLIKASI E-COMMERCE**

- Sebuah perusahaan e-commerce baru saja mengembangkan aplikasi mobile untuk memudahkan customer berbelanja.
- Sebelum aplikasi ini dirilis, tim developer melakukan UAT dengan melibatkan beberapa pengguna yang mewakili pelanggan dari berbagai segmen pasar.
- Mereka akan **menguji proses registrasi akun, pencarian produk, menambah produk ke keranjang, dan checkout**. Jika semua fitur berjalan dengan baik, maka aplikasi siap diluncurkan.

Ilustrasi **Black Box:**

Case Study : APLIKASI E-COMMERCE

No	Fitur	Input	Expected Output	Hasil
1	Login	Email & password valid	Login berhasil	Valid
2	Login	Password salah	Error login	Valid
3	Search	"Laptop"	Produk tampil	Valid
4	Cart	Tambah produk	Masuk keranjang	Valid
5	Checkout	Alamat kosong	Ditolak	Valid

Ilustrasi **White Box:**

Case Study : APLIKASI E-COMMERCE

1. Start
2. Input email & password
3. Cek email terdaftar?
Jika tidak → Error Email,
Jika ya → lanjut
4. Cek password benar?
Jika tidak → Error Password,
Jika ya → Login berhasil
5. End

- 1 → Start
- 2 → Input
- 3 → Cek Email
- 4 → Cek Password
- 5 → Login Berhasil
- 6 → Error Password
- 7 → Error Email

- **Path 1:** 1 → 2 → 3 → 4 → 5
- **Path 2:** 1 → 2 → 3 → 4 → 6
- **Path 3:** 1 → 2 → 3 → 7

Ilustrasi White Box:

Case Study : **APLIKASI E-COMMERCE**

No	Path	Skenario	Input	Expected Output
1	1-2-3-4-5	Email & password benar	email valid, password benar	Login berhasil
2	1-2-3-4-6	Password salah	email valid, password salah	Muncul error password
3	1-2-3-7	Email tidak terdaftar	email tidak valid	Muncul error email



Terima Kasih

GEDUNG
PASCASARJANA
POLITEKNIK NEGERI MALANG